```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import scipy.optimize as sco
import os
from datetime import datetime, timedelta

# Параметры и константы

# Путь к файлу с данными
DATAFILE = r'C:/финансы/data.xlsx'

# Папка для результатов
OUTPUTFOLDER = r'C:/финансы'

# Выбранные активы
SELECTEDINDICESNAMES = ['SBER', 'GAZP', 'MOEX', 'PHOR', 'LKOH',
                        'TATN', 'YNDX', 'MTSS', 'HYDR', 'ROSN']

# Период анализа
STARTDATESTR = '2019-12-04'
FORECASTSTARTDATESTR = '2025-11-01'
ENDDATESTR = '2025-12-04'
FORECASTYEARS = 7

# Ограничения на веса
MINALLOCATION = 0.05
MAXALLOCATION = 0.12

# Параметры Монте-Карло
NUMSIMULATIONS = 30000

# Параметры портфеля (этап 1)
INITIALDEPOSIT = 646_600_000
```

```
ANNUALCONTRIBUTION = 0
RISKAVERSIONDECAY = 0.95

EXPERTFORECASTRETURNS = {
    'SBER': 0.08,
    'GAZP': 0.06,
    'MOEX': 0.07,
    'PHOR': 0.07,
    'HYDR': 0.05,
    'ROSN': 0.06,
    'LKOH': 0.09,
    'MTSS': 0.05,
    'TATN': 0.07,
    'YNDX': 0.10,
}

CUSTOMWEIGHTS = {
    'SBER': 1.8,
    'GAZP': 1.3,
    'MOEX': 1.1,
    'PHOR': 0.8,
    'HYDR': 0.9,
    'ROSN': 0.7,
    'LKOH': 0.9,
    'MTSS': 0.6,
    'TATN': 0.7,
    'YNDX': 0.5,
}

# Параметры портфеля (этап 2)
FORECASTSTARTDATESTR2 = '2032-01-01'
FORECASTYEARS2 = 14
ANNUALCONTRIBUTION2 = -48_000_000
RISKAVERSIONDECAY2 = 0.92
```

```
EXPERTFORECASTRETURNS2 = {
    'SBER': 0.06,
    'GAZP': 0.04,
    'MOEX': 0.05,
    'PHOR': 0.05,
    'HYDR': 0.04,
    'ROSN': 0.04,
    'LKOH': 0.07,
    'MTSS': 0.03,
    'TATN': 0.05,
    'YNDX': 0.07,
}

# Функции

def load_all_sheets(filepath):
    try:
        xls = pd.ExcelFile(filepath)
        print(f"Найдено листов: {len(xls.sheet_names)}")
        print("Первые листы:", xls.sheet_names[:10])

        all_sheets = {}
        for sheet in xls.sheet_names:
            df = pd.read_excel(filepath, sheet_name=sheet)
            all_sheets[sheet] = df

        return all_sheets
    except Exception as e:
        print(f"Ошибка загрузки: {e}")
        return None

def load_data(filepath, ticker_name):
    try:
        xls = pd.ExcelFile(filepath)
```

```
# Сначала пытаемся найти лист с точным именем
sheet_name = None
if ticker_name in xls.sheet_names:
    sheet_name = ticker_name
else:
    # затем ищем по вхождению в название
    for sheet in xls.sheet_names:
        if ticker_name in sheet:
            sheet_name = sheet
            break

if sheet_name is None:
    print(f"Лист для {ticker_name} не найден")
    return None

df = pd.read_excel(filepath, sheet_name=sheet_name, index_col=0, parse_dates=True)
if df.empty:
    print(f"Лист {sheet_name} для {ticker_name} пустой")
    return None

col_name = df.columns[0]
prices = df[col_name].copy()

# Конвертируем цены (формат с запятыми/точками)
def convert_price(x):
    if pd.isna(x):
        return np.nan
    x_str = str(x).replace(' ', '').replace('.', '').replace(',', '.')
    try:
        return float(x_str)
    except Exception:
        return np.nan

prices = prices.apply(convert_price)
prices = prices.dropna()
```

```
        # Удаляем дублирующиеся даты
        prices = prices[~prices.index.duplicated(keep='first')]

        result = pd.DataFrame({ticker_name: prices})
        result.index.name = 'Дата'

        return result

    except Exception as e:
        print(f"Ошибка при загрузке {ticker_name}: {e}")
        return None

def select_indices(df, selected_indices_names):
    available_indices = df.columns.tolist()
    selected_indices = [index for index in selected_indices_names if index in available_indices]

    if not selected_indices:
        print(f"Ни один из {selected_indices_names} не найден. Доступные: {available_indices}")
        return None

    return df[selected_indices]

def select_date_range(df, startdatestr=None, enddatestr=None):
        startdate = datetime.strptime(startdatestr, '%Y-%m-%d').date() if startdatestr else df.index.min().date()
        enddate = datetime.strptime(enddatestr, '%Y-%m-%d').date() if enddatestr else df.index.max().date()

    if startdate <= enddate:
        return df[(df.index.date >= startdate) & (df.index.date <= enddate)]
    else:
        print("Ошибка: начальная дата позже конечной")
        return None
```

```
def calculate_returns(df):
    return df.pct_change(fill_method=None).dropna()

def calculate_annual_stock_returns(returns):
    annual_returns = (1 + returns).groupby(returns.index.year).prod() - 1
    annual_returns.index.name = 'Год'
    return annual_returns

def calculate_annual_portfolio_returns(annual_stock_returns, weights):
    if isinstance(weights, dict):
        w = annual_stock_returns.columns.to_series().map(weights).fillna(0).values
    else:
        w = np.asarray(weights)
    w = w / w.sum()
    return (annual_stock_returns * w).sum(axis=1)

def simulate_future_annual_returns(annual_stock_returns_hist, years=20, num_scenarios=1,
random_state=None):

    rng = np.random.default_rng(random_state)

    hist_values = annual_stock_returns_hist.values.copy()
    n_hist_years, n_assets = hist_values.shape

    # Фильтрация по годам: исключаем годы, где хотя бы одна доходность >100% или < -50%
    mask_rows = np.all((hist_values > -0.5) & (hist_values < 1.0), axis=1)
    hist_values_filtered = hist_values[mask_rows]

    if hist_values_filtered.shape[0] < n_hist_years * 0.3:
        print(f"Внимание: отфильтровано {n_hist_years - hist_values_filtered.shape[0]} год(ов)
как аномальные")

    n_hist_years_clean = hist_values_filtered.shape[0]
    if n_hist_years_clean == 0:
        raise ValueError("После фильтрации не осталось исторических лет для симуляции")
```

```
    sims = np.zeros((num_scenarios, years, n_assets))
    for s in range(num_scenarios):
        idx = rng.integers(0, n_hist_years_clean, size=years)
        sims[s] = hist_values_filtered[idx]

    return sims

def portfolio_annualised_performance(weights, mean_returns, cov_matrix):
    returns = np.sum(mean_returns * weights) * 252
    stddev = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights))) * np.sqrt(252)
    return stddev, returns

def markowitz_optimization(returns, risk_free_rate=0.15):
    mean_returns = returns.mean()
    cov_matrix = returns.cov()
    num_assets = len(returns.columns)

    args_vol = (mean_returns, cov_matrix)
    args_sharpe = (mean_returns, cov_matrix, risk_free_rate)

    constraints = {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}
    bounds_list = tuple((MINALLOCATION, MAXALLOCATION) for _ in range(num_assets))
    initial_guess = np.array([1 / num_assets] * num_assets)

    def minimize_volatility(weights, mean_returns, cov_matrix):
        return portfolio_annualised_performance(weights, mean_returns, cov_matrix)[0]

    def minimize_neg_sharpe(weights, mean_returns, cov_matrix, risk_free_rate):
        p_var, p_ret = portfolio_annualised_performance(weights, mean_returns, cov_matrix)
        if p_var == 0:
            return np.inf
        return -(p_ret - risk_free_rate) / p_var

    print("Оптимизация минимальной волатильности...")
```

```
    opt_vol = sco.minimize(
        minimize_volatility, initial_guess,
        args=args_vol, method='SLSQP',
        bounds=bounds_list, constraints=constraints,
        options={'maxiter': 1000, 'ftol': 1e-9}
    )

    print("Оптимизация максимального Шарпа...")
    opt_sharpe = sco.minimize(
        minimize_neg_sharpe, initial_guess,
        args=args_sharpe, method='SLSQP',
        bounds=bounds_list, constraints=constraints,
        options={'maxiter': 1000, 'ftol': 1e-9}
    )

    if opt_vol.success:
        print("Оптимизация Min Vol успешна")
    else:
        print(f"Min Vol: {opt_vol.message}")

    if opt_sharpe.success:
        print("Оптимизация Max Sharpe успешна")
    else:
        print(f"Max Sharpe: {opt_sharpe.message}")

    return opt_vol, opt_sharpe

def monte_carlo_simulation(returns, num_simulations=100000, risk_free_rate=0.15):
    mean_returns = returns.mean()
    cov_matrix = returns.cov()
    num_assets = len(returns.columns)

    results = np.zeros((4, num_assets + 1, num_simulations))

    for i in range(num_simulations):
```

```
        weights = np.random.uniform(MINALLOCATION, MAXALLOCATION, num_assets)
        weights /= np.sum(weights)

        p_vol, p_ret = portfolio_annualised_performance(weights, mean_returns, cov_matrix)
        sharpe = (p_ret - risk_free_rate) / p_vol if p_vol != 0 else 0

        results[0, 0, i] = p_vol
        results[1, 0, i] = p_ret
        results[2, 0, i] = sharpe
        results[3, :num_assets, i] = weights

    return results

def calculate_max_drawdown(portfolio_values):
    peak = portfolio_values[0]
    max_drawdown = 0

    for value in portfolio_values:
        if value > peak:
            peak = value
        drawdown = (peak - value) / peak
        if drawdown > max_drawdown:
            max_drawdown = drawdown

    return max_drawdown

def plot_portfolio_forecast(portfolio_values, dates, title, output_path):
    plt.figure(figsize=(12, 6))
    plt.plot(dates, np.array(portfolio_values) / 1_000_000, linewidth=2, color='#2E7D8C')
    plt.gca().xaxis.set_major_locator(mdates.AutoDateLocator())
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m-%d'))
    plt.xlabel('Дата')
    plt.ylabel('Портфель (млн руб)')
    plt.title(title, fontsize=14, fontweight='bold')
    plt.grid(True, alpha=0.3)
```

```
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(output_path, dpi=100)
    print(f"График сохранён: {output_path}")
    plt.close()

def plot_efficient_frontier(monte_carlo_results, mean_weights, min_vol_port,
                            max_sharpe_port, returns, output_path):
    plt.figure(figsize=(12, 8))
    plt.scatter(
        monte_carlo_results[0, 0, :] * 100,
        monte_carlo_results[1, 0, :] * 100,
        c=monte_carlo_results[2, 0, :],
        marker='o', cmap='viridis', alpha=0.5
    )
    plt.colorbar(label='Коэффициент Шарпа')
    plt.xlabel('Волатильность (%)', fontsize=11)
    plt.ylabel('Доходность (%)', fontsize=11)
    plt.title('Эффективная граница - Монте-Карло симуляция', fontsize=14, fontweight='bold')

    mean_weights_array = np.array([mean_weights[a] for a in returns.columns])
    mean_vol, mean_ret = portfolio_annualised_performance(mean_weights_array, returns.mean(),
returns.cov())
    plt.scatter(mean_vol * 100, mean_ret * 100, marker='X', color='red', s=200, label='Средний',
zorder=5)

    if min_vol_port.success:
        min_vol, min_ret = portfolio_annualised_performance(min_vol_port.x, returns.mean(),
returns.cov())
        plt.scatter(min_vol * 100, min_ret * 100, marker='*', color='green', s=300, label='Мин. риск',
zorder=5)

    if max_sharpe_port.success:
        max_sharpe_vol, max_sharpe_ret = portfolio_annualised_performance(max_sharpe_port.x,
returns.mean(), returns.cov())
```

```
        plt.scatter(max_sharpe_vol * 100, max_sharpe_ret * 100, marker='P', color='gold', s=200,
                    label='Max Sharpe', zorder=5)

    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(output_path, dpi=100)
    print(f"Граница сохранена: {output_path}")
    plt.close()

def plot_portfolio_structure(weights_history, dates, assets, output_path):
    plt.figure(figsize=(12, 6))
    weights_arr = np.array(weights_history)
    plt.stackplot(dates, weights_arr.T * 100, labels=assets, alpha=0.8)
    plt.gca().xaxis.set_major_locator(mdates.AutoDateLocator())
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m-%d'))
    plt.xlabel('Дата')
    plt.ylabel('Доля в портфеле (%)')
    plt.title('Эволюция структуры портфеля', fontsize=14, fontweight='bold')
    plt.legend(loc='upper left', fontsize=8)
    plt.grid(True, alpha=0.3)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(output_path, dpi=100)
    print(f"Структура портфеля сохранена: {output_path}")
    plt.close()

def rebalance_portfolio(initial_investment, annual_contribution, years, returns_data,
                        custom_weights, risk_aversion_decay=0.95,
                        start_date=None, yearly_portfolio_returns=None):

    custom_assets = returns_data.columns
    custom_weights_array = np.array([custom_weights.get(a, 0) for a in custom_assets])

    if custom_weights_array.sum() > 0:
```

```
    custom_weights_array = custom_weights_array / custom_weights_array.sum()
else:
    # равные веса как запасной вариант
    custom_weights_array = np.array([1 / len(custom_assets)] * len(custom_assets))

portfolio_value = float(initial_investment)
current_weights = custom_weights_array.copy()

mean_returns = returns_data.mean()
asset_volatility = returns_data.std() * np.sqrt(252)
risk_reward = mean_returns / (asset_volatility + 1e-6)
risk_ranking = np.argsort(risk_reward.values)

if start_date is None:
    start_date = returns_data.index.max().date()

dates = [start_date]
portfolio_values = [portfolio_value]
weights_history = [current_weights.copy()]
yearly_results = []

for year in range(years):
    if yearly_portfolio_returns is not None and year < len(yearly_portfolio_returns):
        annual_return = float(yearly_portfolio_returns[year])
    else:
        annual_return = 0.05

    portfolio_value += annual_contribution
    portfolio_value *= (1 + annual_return)

    dates.append(start_date + timedelta(days=365 * (year + 1)))
    portfolio_values.append(portfolio_value)
    yearly_results.append({
        'Год': start_date.year + year + 1,
        'Доходность': f"{annual_return * 100:.2f}%",
```

```
            'Портфель': f"{portfolio_value:,.0f}",
        })

            # Ребалансировка: уменьшаем самый рискованный, увеличиваем 3 наименее рискованных
        riskiest_asset_index = risk_ranking[-1]
        weight_reduction = current_weights[riskiest_asset_index] * (1 - risk_aversion_decay)
        current_weights[riskiest_asset_index] -= weight_reduction

        num_least_risky = min(3, len(risk_ranking))
        least_risky_indices = risk_ranking[:num_least_risky]
        for idx in least_risky_indices:
            current_weights[idx] += weight_reduction / num_least_risky

        current_weights = np.clip(current_weights, 0, 1)
        current_weights = current_weights / np.sum(current_weights)
        weights_history.append(current_weights.copy())

    return portfolio_values, dates, weights_history, yearly_results

# Основная программа

if __name__ == '__main__':
    print("ПОРТФЕЛЬНЫЙ АНАЛИЗ")

    os.makedirs(OUTPUTFOLDER, exist_ok=True)

    print("\nЗагрузка данных...")

    all_sheets = load_all_sheets(DATAFILE)
    if all_sheets is None:
        print("Не удалось загрузить файл")
        raise SystemExit

    all_data = {}
```

```
    for ticker in SELECTEDINDICESNAMES:
        df = load_data(DATAFILE, ticker)
        if df is not None:
            all_data[ticker] = df
        else:
            print(f"Пропускаю {ticker}")

    if not all_data:
        print("Не удалось загрузить данные по акциям")
        raise SystemExit

    df_combined = pd.concat(all_data.values(), axis=1)
    selected_data = select_indices(df_combined, SELECTEDINDICESNAMES)

    if selected_data is None or selected_data.empty:
        print("Не удалось выбрать данные")
        raise SystemExit

    prices_data = select_date_range(selected_data, STARTDATESTR, ENDDATESTR)
    if prices_data is None or prices_data.empty:
        print("Не удалось выбрать период")
        raise SystemExit

    returns = calculate_returns(prices_data)
    print(f"Загружены данные по {len(returns.columns)} акциям")
    print(f"Период: {returns.index.min().date()} - {returns.index.max().date()}")

    annual_stock_returns = calculate_annual_stock_returns(returns)
    print("\nГодовая доходность по акциям:")
    print(annual_stock_returns)

    equal_weights = np.array([1 / len(returns.columns)] * len(returns.columns))
        annual_portfolio_returns = calculate_annual_portfolio_returns(annual_stock_returns,
equal_weights)
    print("\nГодовая доходность портфеля по акциям:")
```

```
    print(annual_portfolio_returns)

    print("\n=== ГЕНЕРАЦИЯ ПРОГНОЗА ГОДОВЫХ ДОХОДНОСТЕЙ ===")
    sims = simulate_future_annual_returns(annual_stock_returns, years=20, num_scenarios=1,
random_state=42)
    future_annual_stock_returns = sims[0]

    w_equal = equal_weights
    future_annual_portfolio_returns = future_annual_stock_returns @ w_equal

    print("Смоделированные годовые доходности портфеля (20 лет):")
    for i, ret in enumerate(future_annual_portfolio_returns):
        print(f"  Год {2025 + i}: {ret * 100:.2f}%")

    print("\n" + "=" * 70)
    print("ЭТАП 1: НАКОПЛЕНИЕ (2025-2032, 7 ЛЕТ)")
    print("=" * 70)
    print(f"Начальный капитал: {INITIALDEPOSIT:,.0f} руб")

    print("\nОптимизация Марковица...")
    markowitz_min_vol, markowitz_max_sharpe = markowitz_optimization(
        returns,
        risk_free_rate=0.15
    )
    print("Оптимизация завершена")

    print("\nМонте-Карло симуляция...")
    monte_carlo_results = monte_carlo_simulation(
        returns,
        num_simulations=NUMSIMULATIONS,
        risk_free_rate=0.15
    )
    mean_weights_mc = np.mean(monte_carlo_results[3, :, :], axis=1)
    mean_weights_mc = np.clip(mean_weights_mc, MINALLOCATION, MAXALLOCATION)
    mean_weights_mc = mean_weights_mc / mean_weights_mc.sum()
```

```
    mean_weights_dict = dict(zip(returns.columns, mean_weights_mc))

    print(f"Выполнено {NUMSIMULATIONS:,} симуляций")

    efficient_frontier_path = os.path.join(OUTPUTFOLDER, 'efficient_frontier_stage1.png')
    plot_efficient_frontier(
        monte_carlo_results,
        mean_weights_dict,
        markowitz_min_vol,
        markowitz_max_sharpe,
        returns,
        efficient_frontier_path
    )

    print("\nРебалансировка портфеля (Этап 1)...")
    forecast_start_date = datetime.strptime(FORECASTSTARTDATESTR, '%Y-%m-%d').date()

    future_annual_portfolio_returns_stage1 = future_annual_portfolio_returns[:FORECASTYEARS]

    portfolio_values_1, dates_1, weights_history_1, yearly_results_1 = rebalance_portfolio(
        INITIALDEPOSIT,
        ANNUALCONTRIBUTION,
        FORECASTYEARS,
        returns,
        CUSTOMWEIGHTS,
        RISKAVERSIONDECAY,
        forecast_start_date,
        yearly_portfolio_returns=future_annual_portfolio_returns_stage1
    )

    stage1_plot_path = os.path.join(OUTPUTFOLDER, 'forecast_stage1.png')
    plot_portfolio_forecast(
        portfolio_values_1,
        dates_1,
```

```
    'Прогноз портфеля (2025-2032)',
    stage1_plot_path
)

structure_plot_path = os.path.join(OUTPUTFOLDER, 'structure_stage1.png')
plot_portfolio_structure(
    weights_history_1,
    dates_1,
    returns.columns,
    structure_plot_path
)

max_drawdown_1 = calculate_max_drawdown(portfolio_values_1)
print(f"Max Drawdown Этап 1: {max_drawdown_1:.2%}")

print("\nИТОГИ ЭТАПА 1:")
for result in yearly_results_1:
   print(f"Год {result['Год']}: Доход {result['Доходность']}, Портфель {result['Портфель']}")

final_portfolio_value_1 = portfolio_values_1[-1]
final_weights_1 = weights_history_1[-1]
print(f"\nФинальная стоимость (2032): {final_portfolio_value_1:,.0f} руб")
print(f"Прирост: {(final_portfolio_value_1 - INITIALDEPOSIT):,.0f} руб "
      f"({((final_portfolio_value_1 / INITIALDEPOSIT - 1) * 100):.2f}%)")

print("\n" + "=" * 70)
print("ЭТАП 2: FIRE (2032-2046, 14 ЛЕТ)")
print("=" * 70)
print(f"Начальный капитал: {final_portfolio_value_1:,.0f} руб")
print(f"Годовые расходы: {abs(ANNUALCONTRIBUTION2):,.0f} руб (4 млн/мес)")
required_capital_fire = 48_000_000 / 0.04
print(f"Правило 4%: нужно {required_capital_fire:,.0f} руб")

      forecast_start_date_2 = datetime.strptime(FORECASTSTARTDATESTR2, '%Y-%m-%d').date()
```

```
# Берём следующие 14 лет (индексы 7–20 включительно → максимум 14 значений)
future_annual_portfolio_returns_stage2 = future_annual_portfolio_returns[FORECASTYEARS:FORECASTYEARS + FORECASTYEARS2]

portfolio_values_2, dates_2, weights_history_2, yearly_results_2 = rebalance_portfolio(
    final_portfolio_value_1,
    ANNUALCONTRIBUTION2,
    FORECASTYEARS2,
    returns,
    dict(zip(returns.columns, final_weights_1)),
    RISKAVERSIONDECAY2,
    forecast_start_date_2,
    yearly_portfolio_returns=future_annual_portfolio_returns_stage2
)

stage2_plot_path = os.path.join(OUTPUTFOLDER, 'forecast_stage2.png')
plot_portfolio_forecast(
    portfolio_values_2,
    dates_2,
    'Прогноз FIRE (2032-2046)',
    stage2_plot_path
)

structure_plot_path_2 = os.path.join(OUTPUTFOLDER, 'structure_stage2.png')
plot_portfolio_structure(
    weights_history_2,
    dates_2,
    returns.columns,
    structure_plot_path_2
)

max_drawdown_2 = calculate_max_drawdown(portfolio_values_2)
print(f"Max Drawdown Этап 2: {max_drawdown_2:.2%}")
```

```
print("\nИТОГИ ЭТАПА 2:")
for i, result in enumerate(yearly_results_2):
    if i % 2 == 0:
        print(f"Год {result['Год']}: Доход {result['Доходность']}, Портфель {result['Портфель']}")

final_portfolio_value_2 = portfolio_values_2[-1]
print(f"\nФинальная стоимость (2046): {final_portfolio_value_2:,.0f} руб")

print("\n" + "=" * 70)
print("ФИНАЛЬНЫЙ АНАЛИЗ")
print("=" * 70)

if final_portfolio_value_2 > required_capital_fire:
    print(f"\n✅ FIRE ДОСТИГНУТ!")
    print(f"Портфель к 2046: {final_portfolio_value_2:,.0f} руб")
    print(f"Требуется: {required_capital_fire:,.0f} руб")
    print(f"Запас: {(final_portfolio_value_2 - required_capital_fire):,.0f} руб")
else:
    print(f"\n❌ FIRE НЕ ДОСТИГНУТ")
    print(f"Портфель к 2046: {final_portfolio_value_2:,.0f} руб")
    print(f"Требуется: {required_capital_fire:,.0f} руб")
    print(f"Не хватает: {(required_capital_fire - final_portfolio_value_2):,.0f} руб")

print(f"\nРезультаты сохранены в: {OUTPUTFOLDER}")
```